

Assignment 06

Multimodal Search System for E-Commerce

Submit V01: TODAY

Before 16:00 (For class 03) and Before 19:00 (For class 04)

Submit V02: Before 23:00 Wednesday (07/10)

Information System Analysis and Design

1 Assignment Overview

Modern e-commerce systems allow customers to search for products, orders, and other information using different interaction modes. A customer may enter a keyword, speak a natural-language query, or upload an image of a product.

The objective of this assignment is to design and implement a small prototype of a **multi-modal e-commerce search system**.

The assignment connects four important topics:

1. software requirements analysis;
2. three-layer software architecture;
3. UML modeling using Visual Paradigm;
4. Python implementation of a search prototype.

The system should support at least the following search modes:

- keyword/text search;
- voice search simulated by speech-to-text output;
- image-based product search.

The central design principle is:

Different Inputs $\longrightarrow$ Common Query Representation $\longrightarrow$ Retrieval $\longrightarrow$ Ranking $\longrightarrow$ Results. (1)

The assignment does not require a production e-commerce system. The objective is to demonstrate the relationship between system design, UML modeling, and an executable AI-oriented prototype.

2 Learning Objectives

After completing this assignment, students should be able to:

1. identify functional requirements of an e-commerce search system;
2. identify actors and use cases;

3. design a three-layer software architecture;
4. construct UML diagrams using Visual Paradigm;
5. distinguish the presentation, application, and data layers;
6. design a common interface for text, voice, and image search;
7. implement a simple product-search service in Python;
8. implement a basic image-similarity search prototype;
9. explain how AI components can be incorporated into a conventional software architecture.

3 Problem Statement

Consider an online shopping system containing products, customers, and orders.

A customer may want to perform queries such as:

- “black running shoes”;
- “find Nike shoes under 100 dollars”;
- “show me products similar to this image”;
- “find my order 20261001”;
- “where is my latest order?”

The system should accept different forms of input.

Table 1: Supported search modalities

Mode	Input	Processing
Text	Keyword or natural language	Text normalization and retrieval
Voice	Audio query	Speech-to-text followed by text retrieval
Image	Product photograph	Image representation followed by similarity search

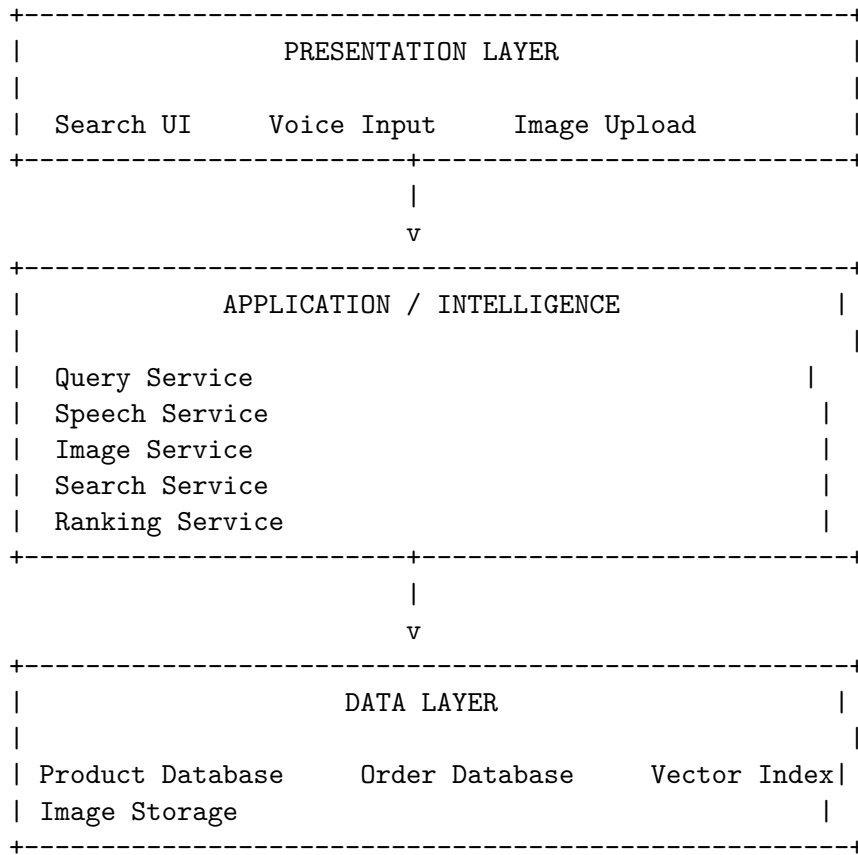
For the prototype, students may simulate speech recognition by providing the transcribed text directly.

4 System Architecture

The proposed system uses three layers:

1. Presentation Layer;
2. Application / Intelligence Layer;
3. Data Layer.

The architecture is:



The three layers have different responsibilities.

Table 2: Responsibilities of the three layers

Layer	Responsibility
Presentation	Collect user input and display search results.
Application / Intelligence	Interpret queries, convert modalities, retrieve candidates, filter candidates, and rank results.
Data	Store products, orders, images, metadata, and vector representations.

5 Layer 1: Presentation Layer

The Presentation Layer provides interaction between the customer and the system.

The main components are:

- SearchUI;
- VoiceInput;
- ImageUpload;
- SearchResultView.

A simplified interaction is:

Customer

```

|
+----- Text / Keyword
|
+----- Voice
|
+----- Image
|
v
Search UI
|
v
Application Layer

```

The Presentation Layer should not directly access the database.
For example, the following design should be avoided:

SearchUI --> ProductDatabase

Instead:

```

SearchUI
|
v
SearchService
|
v
ProductRepository
|
v
ProductDatabase

```

This separation makes the system easier to maintain and extend.

6 Layer 2: Application and Intelligence Layer

The Application Layer contains the main system logic.

A recommended design is:

```

+-----+
| Application / Intelligence Layer |
|                                   |
| QueryService                    |
| SpeechService                   |
| ImageService                    |
| SearchService                   |
| RankingService                  |
+-----+

```

6.1 Query Service

The Query Service converts different input types into a common query representation.

Let

q_t

denote a text query,

q_v

a voice query, and

q_i

an image query.

The system performs

$$q_t \mapsto z_t, \quad q_v \mapsto z_v, \quad q_i \mapsto z_i, \quad (2)$$

where z_t , z_v , and z_i are representations used by the retrieval system.

For the simple prototype, the representation may be a Python dictionary.

```
{
  "type": "text",
  "query": "black running shoes"
}
```

6.2 Voice Service

The voice-search pipeline is:

```
Microphone
|
v
Audio
|
v
Speech-to-Text
|
v
Text Query
|
v
Search Service
```

For the first prototype, students may simulate the speech-to-text component:

```
voice_text = "find black running shoes"

query = {
  "type": "voice",
  "query": voice_text
}
```

Students who want to extend the project may replace the simulated component with an actual speech-recognition system.

6.3 Image Service

Image search uses an image representation.

Let

$$f_{\theta}(I)$$

denote an image encoder, where I is an input image and $f_{\theta}(I)$ is its feature vector.
For an image query I_q ,

$$z_q = f_{\theta}(I_q). \quad (3)$$

Each product image has an embedding

$$z_p = f_{\theta}(I_p).$$

A simple similarity measure is cosine similarity:

$$s(I_q, I_p) = \frac{z_q^T z_p}{\|z_q\| \|z_p\|}. \quad (4)$$

Products with larger similarity values are returned first.

6.4 Search Service

The Search Service retrieves candidate products.

For a text query, a simple prototype can use keyword matching.

For an image query, it can use vector similarity.

The general retrieval operation is

$$\mathcal{R}(q) = \text{TopK}_{x \in \mathcal{X}} s(q, x), \quad (5)$$

where $\mathcal{X}$ is the product collection.

6.5 Ranking Service

After retrieval, the system may rank candidates using several signals.

For example,

$$S(x|q) = \alpha S_{\text{text}} + \beta S_{\text{image}} + \gamma S_{\text{business}}. \quad (6)$$

Here:

- S_{text} measures textual similarity;
- S_{image} measures visual similarity;
- S_{business} may consider price, stock, popularity, or other application-specific information.

For Assignment 05, students only need to implement a simple ranking mechanism.

7 Layer 3: Data Layer

The Data Layer stores the information used by the system.

A minimal system contains:

Product Database

Order Database

Image Storage

Vector Index

A simplified Product entity is:

Product

product_id

name

category

color

price

stock

description

image

embedding

A simplified Order entity is:

Order

order_id

customer_id

date

status

total

An OrderItem entity may contain:

OrderItem

order_id

product_id

quantity

price

8 Visual Paradigm Modeling

Students must use Visual Paradigm to construct at least three UML diagrams.

8.1 Diagram 1: Use Case Diagram

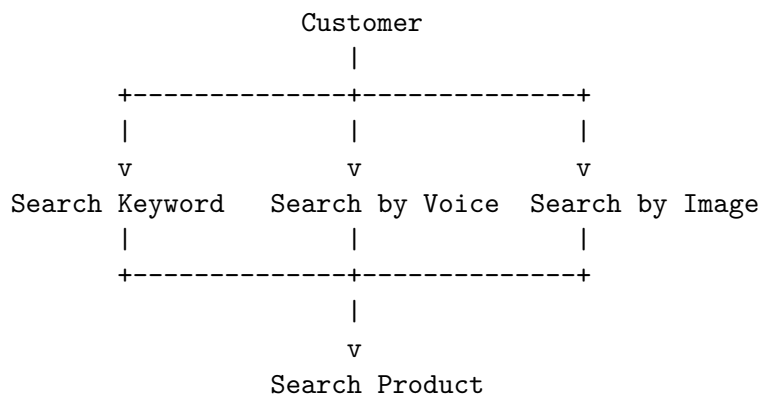
Create an actor:

Customer

The main use cases are:

- Search Product;
- Search by Keyword;
- Search by Voice;
- Search by Image;
- Search Order;
- View Product;
- View Order.

A simplified structure is:



The student should create the diagram using UML notation rather than simply drawing rectangles.

8.2 Diagram 2: Three-Layer Component Diagram

Create three packages:

1. Presentation Layer;
2. Application / Intelligence Layer;
3. Data Layer.

Recommended components are:

Presentation:

SearchUI
VoiceInput
ImageUpload
SearchResultView

Application:

QueryService
SpeechService
ImageService
SearchService
RankingService

Data:

ProductRepository
OrderRepository
VectorIndex
ProductDatabase
OrderDatabase
ImageStorage

The dependencies should follow:

Presentation $\rightarrow$ Application $\rightarrow$ Data. (7)

Avoid direct dependencies from Presentation to Data.

8.3 Diagram 3: Sequence Diagram

Create a sequence diagram for voice search.

The recommended participants are:

Customer
SearchUI
SpeechService
QueryService
SearchService
ProductRepository
RankingService

The sequence is:

1. Customer provides voice input.
2. SearchUI sends audio to SpeechService.
3. SpeechService returns text.
4. QueryService constructs a query representation.
5. SearchService retrieves products.
6. ProductRepository accesses product data.
7. RankingService ranks candidates.
8. SearchUI displays the results.

9 Python Prototype

The Python prototype should mirror the architecture created in Visual Paradigm.

The recommended directory structure is:

```
assignment05/  
|  
+-- main.py  
|  
+-- presentation/
```

```

|   +--- search_ui.py
|
+--- application/
|   +--- query_service.py
|   +--- speech_service.py
|   +--- image_service.py
|   +--- search_service.py
|   +--- ranking_service.py
|
+--- data/
|   +--- product_repository.py
|   +--- order_repository.py
|   +--- vector_index.py
|
+--- data/
    +--- products.json
    +--- images/

```

10 Step 1: Product Repository

Create the file:

data/product_repository.py

Use the following example:

```

class ProductRepository:

    def __init__(self):

        self.products = [
            {
                "id": 1,
                "name": "Nike Running Shoes",
                "category": "shoes",
                "color": "black",
                "price": 120,
                "stock": 10
            },
            {
                "id": 2,
                "name": "Adidas Running Shoes",
                "category": "shoes",
                "color": "white",
                "price": 100,
                "stock": 15
            },
            {
                "id": 3,
                "name": "Black Leather Bag",
                "category": "bag",
                "color": "black",
                "price": 80,
                "stock": 20
            },
            {

```

```

        "id": 4,
        "name": "Blue Sports Shoes",
        "category": "shoes",
        "color": "blue",
        "price": 75,
        "stock": 12
    }
]

def all_products(self):
    return self.products

```

Students should add at least ten products.

11 Step 2: Query Service

Create:

application/query_service.py

```

class QueryService:

    def text_query(self, text):

        return {
            "type": "text",
            "query": text
        }

    def voice_query(self, text):

        return {
            "type": "voice",
            "query": text
        }

    def image_query(self, embedding):

        return {
            "type": "image",
            "embedding": embedding
        }

```

The important architectural idea is that all modalities are converted into a common query object.

12 Step 3: Text Search

Create:

application/search_service.py

```

class SearchService:

    def __init__(self, repository):

```

```

        self.repository = repository

    def search_text(self, query):

        query = query.lower()

        results = []

        for product in self.repository.all_products():

            text = (
                product["name"] + " " +
                product["category"] + " " +
                product["color"]
            ).lower()

            words = query.split()

            score = 0

            for word in words:

                if word in text:
                    score += 1

            if score > 0:

                results.append(
                    (product, score)
                )

        results.sort(
            key=lambda x: x[1],
            reverse=True
        )

        return results

```

This is intentionally simple.

The purpose is to demonstrate the architecture before introducing more advanced AI models.

13 Step 4: Voice Search

For Assignment 05, voice recognition may be simulated.

Create:

application/speech_service.py

```

class SpeechService:

    def transcribe(self, audio_input):

        # Prototype:
        # audio_input is already the transcribed text.

        return audio_input

```

Then:

```
speech_service = SpeechService()

text = speech_service.transcribe(
    "find black running shoes"
)

results = search_service.search_text(text)
```

Students should clearly explain in the report that this is a simulation of speech recognition.

14 Step 5: Image Similarity

For the first prototype, students may use artificial feature vectors.

For example:

```
import numpy as np

product_embeddings = {
    1: np.array([0.9, 0.1, 0.2]),
    2: np.array([0.2, 0.8, 0.1]),
    3: np.array([0.1, 0.2, 0.9]),
    4: np.array([0.8, 0.2, 0.2])
}
```

Define cosine similarity:

```
def cosine_similarity(a, b):

    numerator = np.dot(a, b)

    denominator = (
        np.linalg.norm(a) *
        np.linalg.norm(b)
    )

    if denominator == 0:
        return 0.0

    return numerator / denominator
```

Image search:

```
def search_image(query_embedding,
                 product_embeddings):

    scores = []

    for product_id, embedding in \
        product_embeddings.items():

        score = cosine_similarity(
            query_embedding,
            embedding
        )

        scores.append(
            (product_id, score)
```

```

    )

    scores.sort(
        key=lambda x: x[1],
        reverse=True
    )

    return scores

```

Students may later replace these artificial vectors with embeddings generated by an actual image model.

15 Step 6: Main Program

Create:

main.py

```

from data.product_repository import ProductRepository
from application.query_service import QueryService
from application.search_service import SearchService
from application.speech_service import SpeechService

repository = ProductRepository()

query_service = QueryService()

search_service = SearchService(
    repository
)

speech_service = SpeechService()

print("=== E-Commerce Search Demo ===")

print("\nText search:")

query = query_service.text_query(
    "black shoes"
)

results = search_service.search_text(
    query["query"]
)

for product, score in results:

    print(
        product["name"],
        "| score =",
        score
    )

print("\nVoice search:")

```

```

audio = "find running shoes"

text = speech_service.transcribe(
    audio
)

query = query_service.voice_query(
    text
)

results = search_service.search_text(
    query["query"]
)

for product, score in results:

    print(
        product["name"],
        "| score =",
        score
    )

```

16 Expected Demonstration

The program should demonstrate at least two successful search modes.

For example:

```
=== E-Commerce Search Demo ===
```

Text search:

```
Nike Running Shoes | score = 2
```

```
Blue Sports Shoes  | score = 1
```

Voice search:

```
Nike Running Shoes | score = 1
```

```
Adidas Running Shoes | score = 1
```

```
Blue Sports Shoes | score = 1
```

The exact output depends on the student's product database.

17 Extension: Multimodal Search

Students who complete the basic prototype should implement a multimodal query.

Suppose the customer provides:

- a text query;
- an image;
- optionally a voice query.

The system can combine the representations:

$$z = \lambda_t z_t + \lambda_i z_i + \lambda_v z_v, \quad (8)$$

where

$$\lambda_t + \lambda_i + \lambda_v = 1. \quad (9)$$

For example,

$$\lambda_t = 0.5, \quad \lambda_i = 0.5.$$

A combined ranking function can be:

$$S(p|q) = \lambda_t S_t(p|q_t) + \lambda_i S_i(p|q_i). \quad (10)$$

This produces a simple multimodal retrieval system.

18 Extension: Order Search

The system should also support order search.

A simple repository is:

```
class OrderRepository:

    def __init__(self):

        self.orders = [
            {
                "order_id": "0001",
                "customer_id": "C001",
                "status": "Shipped",
                "total": 150
            },
            {
                "order_id": "0002",
                "customer_id": "C002",
                "status": "Delivered",
                "total": 80
            }
        ]

    def find_order(self, order_id):

        for order in self.orders:

            if order["order_id"] == order_id:
                return order

        return None
```

A customer can therefore ask:

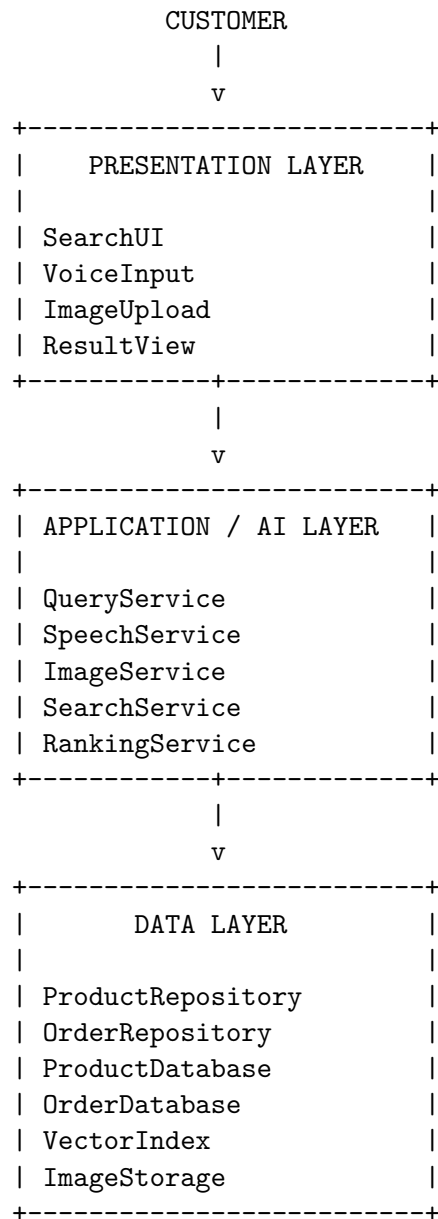
Find order 0001

and the system returns:

```
Order: 0001
Status: Shipped
Total: 150
```

19 Required UML Architecture

The final UML architecture should conceptually resemble:



20 Assignment Tasks

Students must complete the following tasks.

Task 1: Requirements Analysis

Write a short description of the system.

Identify:

- actors;
- functional requirements;
- non-functional requirements;

- input modalities;
- search outputs.

At least five functional requirements are required.

Task 2: Use Case Diagram

Create a UML Use Case Diagram in Visual Paradigm.

The diagram must contain:

- Customer;
- Search Product;
- Search by Keyword;
- Search by Voice;
- Search by Image;
- Search Order;
- View Product;
- View Order.

Task 3: Three-Layer Architecture

Create the three-layer component/package diagram.

Clearly identify:

1. Presentation Layer;
2. Application / Intelligence Layer;
3. Data Layer.

The dependencies between layers must be clear.

Task 4: Sequence Diagram

Create a sequence diagram for:

Customer searches for a product using voice.

The sequence must contain at least:

Customer
SearchUI
SpeechService
QueryService
SearchService
ProductRepository
RankingService

Task 5: Python Implementation

Implement:

- product repository;
- text search;
- simulated voice search;
- image-similarity search;
- result ranking.

Task 6: Demonstration

Demonstrate at least three queries:

1. text query;
2. voice query;
3. image query.

For each query, show:

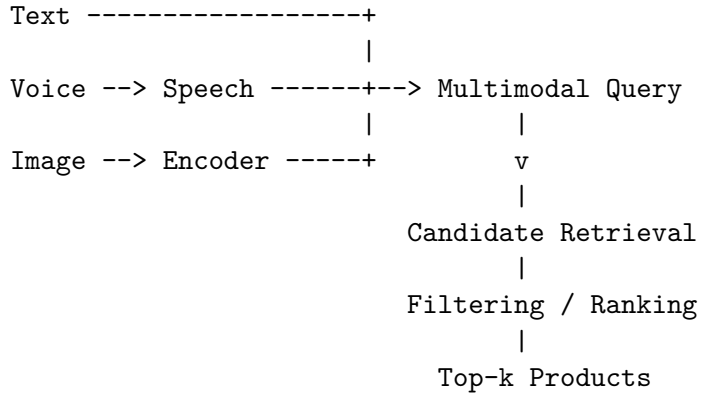
- input;
- processing;
- returned products;
- ranking score.

21 Optional Advanced Work

Students seeking additional credit may implement one or more of the following:

1. real speech-to-text;
2. real image embeddings;
3. vector database;
4. semantic text embeddings;
5. multimodal fusion;
6. product filtering by price;
7. product filtering by category;
8. stock-aware ranking;
9. order-status search;
10. a web-based user interface.

A possible advanced architecture is:



22 Experimental Evaluation

Students should evaluate the prototype using a small test set.

For example:

Table 3: Example evaluation table

Query	Mode	Top-1 Result	Success
black shoes	Text	Nike Running Shoes	Yes
running shoes	Voice	Adidas Running Shoes	Yes
shoe image	Image	Nike Running Shoes	Yes
order O001	Text	O001	Yes

Students should report at least:

- number of test queries;
- number of successful queries;
- success rate;
- examples of incorrect results.

The success rate is

$$\text{Success Rate} = \frac{\text{Number of Successful Queries}}{\text{Total Number of Queries}}. \quad (11)$$

23 Report Structure

The submitted report should contain approximately 10–12 pages.

Recommended structure:

1. Introduction;
2. Problem Description;
3. Requirements Analysis;
4. Use Case Model;
5. Three-Layer Architecture;

6. UML Sequence Diagram;
7. Python Implementation;
8. Multimodal Search Method;
9. Experimental Results;
10. Discussion;
11. Conclusion.

The report should contain screenshots of the Visual Paradigm diagrams and screenshots of the Python program running.

24 Submission Requirements

Each student should submit:

1. PDF report;
2. Visual Paradigm project;
3. Python source code;
4. product dataset;
5. sample images;
6. README file;
7. demonstration results.

The README should explain:

- Python version;
- required packages;
- how to run the program;
- project directory structure;
- example commands;
- limitations of the prototype.

25 Assessment Rubric

The assignment may be assessed using the following rubric.

Table 4: Assessment rubric

Component	Points
Requirements analysis	10
Use Case Diagram	15
Three-Layer Architecture	20
Sequence Diagram	10
Python implementation	20
Multimodal search	10
Experimental evaluation	5
Report quality	5
Demonstration	5
Total	100

26 Important Design Principles

Students should remember the following principles.

Principle 1: Separate UI from Business Logic

The UI should not directly access the database.

$$\text{UI} \not\rightarrow \text{Database.} \quad (12)$$

Instead:

$$\text{UI} \rightarrow \text{Application} \rightarrow \text{Data.} \quad (13)$$

Principle 2: Different Inputs, Common Representation

Voice, text, and image inputs should eventually be transformed into representations that the retrieval system can process.

$$\{\text{text, voice, image}\} \rightarrow \text{query representation.} \quad (14)$$

Principle 3: Retrieval and Ranking Are Different

Retrieval generates candidate products:

$$\mathcal{C}(q).$$

Ranking orders the candidates:

$$\text{Rank}(\mathcal{C}(q)).$$

Students should not treat these two operations as the same.

Principle 4: Start Simple

The first implementation does not need a sophisticated AI model.

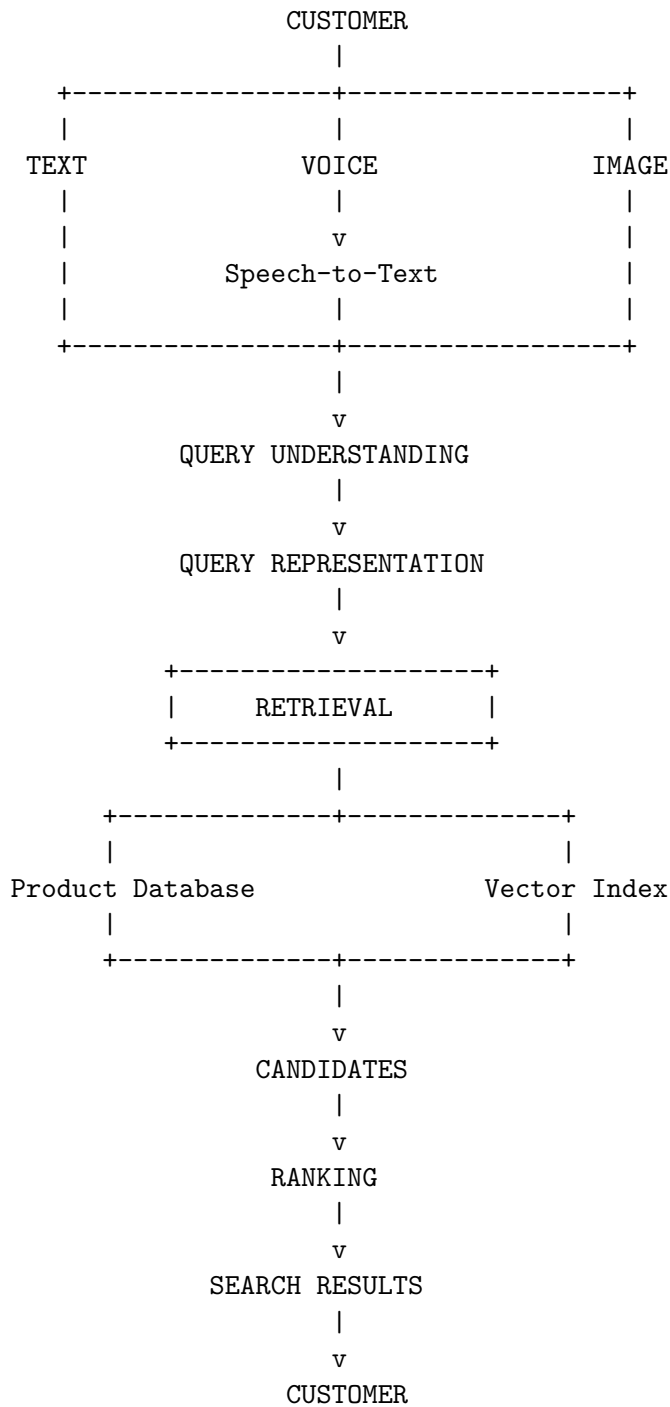
A valid progression is:

$$\text{Keyword Matching} \rightarrow \text{Embedding Search} \rightarrow \text{Multimodal Retrieval.}$$

The objective is to understand the architecture before optimizing the AI component.

27 Final System View

The complete conceptual system is:



The central lesson of Assignment 05 is that an AI-enabled e-commerce system is not only an AI model. It is a complete software system in which:

$$\boxed{\text{User Interface} \rightarrow \text{AI/Application Logic} \rightarrow \text{Data}} \quad (15)$$

and multimodal input is transformed into a unified search process.